Technical Design Document
Team Olive

Contents

Introduction and Overview 3

Purpose and Goals 3

Scope and Target Platform 3

Limitations 3

System Architecture 4

Data Flow 4

GameObject Archetypes 4

Player Behavior 4

Main Loop of Engine 5

Core Subsystems and Components 5

Rendering System 5

Tilemap System 6

Physics System 6

Audio System 6

Animation System 7

Action System 7

Resource Management 7

Interfaces and APIs 8

class ScriptBase 8

Implementation Considerations 8

Documentation 9

Introduction and Overview

Mitosis Engine is a modular, component-based 2D game engine built in modern C++ (C++23). The engine provides a comprehensive suite of systems for rendering, physics simulation, audio playback, animation, and gameplay scripting. Designed with extensibility and performance in mind, it supports multiple standalone game projects while maintaining a clean separation between engine core and application code.

Purpose and Goals

The primary objectives of Mitosis Engine include:

- Provide a robust foundation for 2D game development with emphasis on platformer and action games.
- Enable rapid prototyping through data-driven design and JSON-based serialization.
- Maintain clean architecture through interface abstraction and dependency injection.
- Provide necessary features including physics, audio, and animation systems.

Scope and Target Platform

Platform Support: Windows and Linux (x64 architecture). The engine targets mid-range hardware with a performance goal of 60 FPS for typical 2D games.

Primary Use Cases: 2D side-scrolling platformers, action games, and rhythm-based games. The architecture is intentionally designed to be extensible for other 2D game genres.

Limitations

The engine does not provide functionalities for 3D development.

There is no support for network within the engine. It cannot handle online game development.

Only limit light database functions are supported by UUID search and serialization of entities. No further database implementation to allow mathematical analysis of player behavior or distributive synchronization of states

Physics system is lack of friction, restitution. It requires a more advance implementation.

Audio system relies on the functionalities of the FMOD, which may bring extra workloads for producing audio assets handling unexpected behavior from FMOD.

System Architecture

Mitosis Engine employs a component-based architecture where GameObjects serve as containers for Components. This design pattern provides several key advantages:

- Flexibility - behaviors can be composed from reusable components
- Modularity - systems are decoupled and communicate through well-defined interfaces
- Data-driven design - game objects can be defined in JSON and instantiated at runtime
- Extensibility - new components can be added without modifying core engine code

The engine supports multiple standalone application projects, each linking against shared engine libraries:

- Mitosis-Engine (static library) - core engine functionality
- Mitosis-Launcher (static library)
- App Projects (console executables) - individual games with custom scenes and scripts

Each app provides an application subclass, initial Scene, and CreateApplication() entry point. This architecture enables teams to work on different games simultaneously while sharing the same engine infrastructure.

Data Flow

GameObject Archetypes

The archetype of a GameObject is lazy loaded in the `GameObjectFactory.h`. The configuration of a GameObject is in a Json file. When it is requested for the first time, the factory will search for the Json file with corresponding name and start building an archetype from it. Once it finishes building, the archetype will be stored in the container and if the archetype is requested again, the factory will call a copy constructor and reduce the cost of reading in a new GameObject from the very beginning.

Player Behavior

The behavior of the player (typically input event, starting application, shutdown application and so on) is handled by the main loop. All inputs are consumed first in a scene, and the system will record the state of the input system, all other components can check whether it is triggered.

Main Loop of Engine

The engine follows a standard game loop pattern:

1. The app will initialize first. The scene system, resource manager, and debug drawer will be initialized.
2. In the running phase of the application, first of all the scene system will be updated, calling all the update functions in the GameObjects and components of the scene. Then it is the LateUpdate function to modify object state if demanded.
3. In the render phase, render functions in all GameObjects and Components will be called first, then it's the DrawSprite function from Renderer. The debug drawer is called at the end of render phase.

Components communicate through GameObjects references and callbacks, and GameObjects communicate by accessing another one and the same scene. The lifecycle of a GameObject starts from a scene where it's created and ended when the Destroy function is called on purpose during gameplay logic or the scene is closed without setting the `DontDestory` flag true.

Core Subsystems and Components

Rendering System

Responsibilities: Sprite rendering, mesh management, shader compilation, texture loading, camera transformations

Architecture: The rendering system uses OpenGL through SDL2. Meshes define vertex data with UV coordinates configured for intuitive top-down coordinate systems (y-up). Shaders are loaded via the Resource Manager and compiled at runtime.

These are the classes that plays an important role in rendering part.

- SpriteRenderer: Renders 2D sprites by managing the rendering pipeline for sprite-based game objects. It doesn't render directly but is called by the main Renderer system.
- Renderer: This is a classic 2D rendering system that handles sprite rendering with proper transparency sorting and OpenGL state management, ensuring correct visual output for both opaque and transparent sprites/particles.

Tilemap System

Responsibilities: Orthogonal tilemap rendering, tile property management, chunk-based batching, collider generation.

Architecture: Three-layer design: Tilemap, Tileset, and Tile. The Tilemap component owns a ChunkManager for asynchronous mesh generation and a TilemapRenderer for efficient rendering of visible chunks.

Feature:

- Grid-based tile placement with SetTile(x, y, id) API
- Tile properties defined in JSON (collidable flag, visual properties)
- Automatic collider generation based on tile properties
- Chunk-based rendering for large tile maps

Physics System

Responsibilities: 2D rigid body dynamics, collision detection, collision resolution, event dispatch.

Architecture: User can actively apply the command-driven behaviors to the GameObject where gameplay requests physical actions through components, or add physics related attribute that will be automatically updated each frame and finally PhysicsSystem2D will tackle with them during fixed update.

The 2D rigid body component can add physics effects on a GameObject. Different body types will react differently to physics.

The engine allows setting collider as a trigger, enabling triggering some logic of the game without performing physics features. The collision is handled by the physics system once in each engine loop. It checks the collision between triggers and then checks the collision involving colliders.

Audio System

Responsibilities: Audio playback, mixing, spatial audio, bus control, snapshot management.

Architecture: Backend-agnostic design with FMOD Studio API as the primary implementation. The system cleanly separates engine-level audio logic from platform/middleware implementations.

The features of the audio system include:

- Bank-based content management
- Event-based sound playback with opaque handles
- 2D spatial audio (3D-ready attributes for future expansion)
- Bus volume and pause control
- Snapshot system for dynamic mixing
- CI-safe null backend for headless builds

Animation System

Responsibilities: State-based sprite animation, frame sequencing, state transitions.

Architecture: The system supports multiple animation states (Idle, Walk, etc.) defined in JSON. State changes are triggered via ChangeState() calls. Requires both SpriteRenderer and Animation components.

Action System

Responsibilities: Time-based behaviors, animation curves, sequential and parallel execution.

Available Actions:

- TranslateAction - Move to target position
- RotateAction - Rotate to target rotation
- ScaleAction - Scale to target size

Features:

- Easing functions for smooth interpolation
- Delay parameter for timed sequences
- Blocking flag for sequential execution
- Group IDs for selective blocking
- Runtime time control (pause, fast-forward, slow-down)

Resource Management

Responsibilities: Asset loading, caching, lifetime management for textures, shaders, and meshes.

Architecture: Static ResourceManager class with lazy loading. Resources are created on first request and cached for subsequent requests. Default resources are provided when no path is specified.

Interfaces and APIs

There are two ways users can interact with the engine under the current implementation. First is using the components provided by the engine, which is briefly discussed in the previous chapter, and the other one is adding a ScriptBase class as the parent for user-defined script.

class ScriptBase

This is a class responsible for authoring gameplay logic, Unity-style life cycle callbacks, decoupled behavior from engine core and allowing user to start working on their own gameplay logic instead of checking what's the code within the engine.

Supported Callbacks:

- OnCreate, OnUpdate, OnLateUpdate
- OnEnable, OnDisable
- OnCollisionEnter2D, OnCollisionStay, OnCollisionExit
- DeserializeFields (for JSON field loading)

Scripts are native C++ with no reflection system or runtime overhead. The factory pattern enables data-driven script attachment via JSON. Scripts have full access to engine APIs while remaining completely decoupled from engine headers.

Implementation Considerations

Existing Optimization Strategies:

- Tilemap chunk-based batching reduces draw calls
- Resource caching prevents redundant loads
- Fixed timestep physics for consistent simulation
- Memory tracking via custom allocators
- FMOD event instance cleanup per-frame

Project Building:

Premake Configuration: The engine uses Premake 5 for cross-platform project generation. The build system supports three dependency types:

- Header-Only Libraries - GLM, spdlog (included via paths)
- Prebuilt Libraries - GLFW (with DLL copy steps)
- CMake Source Libraries - Built via scripts, installed to third-party/install/

- vcpkg package manager - boost/uuid, boost/random

FMOD Integration: FMOD is an external SDK configured via FMOD_*SDK environment variable. If not present, the build falls back to NullAudioBackend*.

Documentation

More information on the implementation and design choices can be found as part of the wiki that is present in the repository: https://github.com/DigiPen-GAM5xx/gam541-olive/wiki.